An Introduction to FlashAttention

Chih-Jen Lin
National Taiwan University, MBZUAI

Last updated: October 19, 2025

Inefficiency of Attention Operation I

- Similar to the memory-access issue discussed above for matrix-matrix products, a possible bottleneck of attention is on moving data (i.e., matrices) between lower-level and upper-level memory on GPUs.
- To analyze this issue, we must check the number of memory accesses.

Attention Operations I

- For the discussion, first we recall details of attention. For simplicity, we consider the single-head attention.
- If the input matrix is

$$\tilde{Z} \in \mathbf{R}^{T \times d},$$

the attention operation is

$$\text{SoftMax}\left(\frac{\tilde{Z}W_Q W_K^\top (\tilde{Z})^\top}{\sqrt{d}}\right)\tilde{Z}W_V. \tag{1}$$

Attention Operations II

- We consider three trainable weight matrices

$$W_Q \in \mathbf{R}^{d\times d}, W_K \in \mathbf{R}^{d\times d}, W_V \in \mathbf{R}^{d\times d}$$

to convert the input matrix $\tilde{Z}$ to

$$\tilde{Z}W_Q \in \mathbf{R}^{T\times d}, \quad \tilde{Z}W_K \in \mathbf{R}^{T\times d}, \quad \tilde{Z}W_V \in \mathbf{R}^{T\times d}.$$

Attention Operations III

- In (1), the SoftMax function is applied on each row $\boldsymbol{z}$ of an input matrix in the following way.

$$\text{SoftMax}(\boldsymbol{z}) = \begin{bmatrix} \frac{\exp(z_1)}{\sum_j \exp(z_j)} \\ \vdots \\ \frac{\exp(z_T)}{\sum_j \exp(z_j)} \end{bmatrix}. \tag{2}$$

Attention Operations IV

- FlashAttention (Dao et al., 2022) defines that

$$\mathbf{Q} := \tilde{Z}W_Q, \quad \mathbf{K} := \tilde{Z}W_K, \quad \mathbf{V} := \tilde{Z}W_V,$$

and assumes that $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ had been already precomputed.

- Omitting $1/\sqrt{d}$ for simplification, FlashAttention turns (1) into

$$\mathbf{O} := \text{SoftMax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}, \tag{3}$$

where $\mathbf{O} \in \mathbf{R}^{T\times d}$ is the output matrix of the attention.

Memory Accesses in Attention I

- We still assume that our machine has only two layers of memory:
  - main memory, and
  - secondary memory.
- If an operand is not available in main memory, we must transport it from secondary memory.
- Now consider (3) and check intermediate values during computation.

Memory Accesses in Attention II

- We need

$$\mathbf{Q}\mathbf{K}^\top \in R^{T\times T}, \tag{4}$$

$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top) \in R^{T\times T}, \tag{5}$$

$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V} \in R^{T\times d}. \tag{6}$$

- As in general

$$T \gg d,$$

even though the output

$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V} \in R^{T\times d}$$

Memory Accesses in Attention III

is smaller, storing $T \times T$ intermediate matrices is the main difficulty.

Insufficient Memory to Store $T \times T$ Intermediate Matrices I

- Our first analysis is to assume that

$$T \times T$$

  matrices cannot be stored in the main memory, and check the need to move these matrices
- If we consider (4)-(6) as independent operations, immediately we see the following major memory accesses:

Insufficient Memory to Store $T \times T$ Intermediate Matrices II

- write

$$\mathbf{Q}\mathbf{K}^\top \in R^{T \times T} \tag{7}$$

  to secondary memory,
- load the matrix (7) from secondary memory to calculate

$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top) \tag{8}$$

  and write results back to secondary memory, and

Insufficient Memory to Store $T \times T$ Intermediate Matrices III

- load the matrix in (8) for calculating

$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V} \in R^{T\times d}.$$

- We assume that even though storing an $T \times T$ matrix in main memory is not possible, the computer has a way to sequentially work on part of the input data in main memory and gradually generate/save part of the whole output to secondary memory

Insufficient Memory to Store $T \times T$ Intermediate Matrices IV

It is just like that we do matrix-matrix products all the time, but never worry that our highest-level memory (i.e., registers) is insufficient to store operands.

- Now we conclude that a naive implementation of attention leads to

$$4 \times T^2 \tag{9}$$

accesses between main and secondary memory.

Memory Versus Computation I

- We see attention involves the following operations and list their respective cost.

$$\mathbf{Q}\mathbf{K}^\top : 2T^2d,$$
$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top) : 3T^2,$$
$$\text{SoftMax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V} : 2T^2d.$$

Memory Versus Computation II

- Clearly, if

$$4T^2d \times \text{ cost per operation}$$
$$<$$
$$4T^2 \times \text{ cost per memory access},$$

  then attention is memory bounded.
- Example:
  - Consider $T = 1024$ and $d = 64$, running on an A100 80GB SXM GPU.

Memory Versus Computation III

- This GPU sustains up to 312 tera floating point operations per second (TFLOPS) for half-precision floating-point format (FP16) operations.
- The GPU is equipped with 80 GB of high-bandwidth memory (HBM), corresponding to the secondary memory in our setting and offering a memory bandwidth of 2.04 tera bytes per second (TB/s).

Memory Versus Computation IV

- Then, the total compute cost is

$$\frac{4 \times 1024^2 \times 64 \text{ FLOPs}}{312 \text{ TFLOPS}} \approx 0.86 \ \mu\text{s}.$$

- The total memory access cost is

$$\frac{4 \times 1024^2 \times 2 \text{ bytes}}{2.04 \text{ TB/s}} \approx 3.74 \ \mu\text{s}.$$

  Note that we assume each value is stored in FP16 (two bytes).

- To accelerate attention, we must reduce the number of memory accesses.

Situation When Storing $T \times d$ Matrices Are Possible I

- Now we assume that the main memory is sufficient to store

$$T \times d \text{ matrices such as } Q, K, \text{ and } V.$$

- Let us develop strategies to reduce the number of memory accesses in (9).
- In particular, we would like to avoid loading and storing intermediate results.

Situation When Storing $T \times d$ Matrices Are Possible II

- That is, we should generate the attention results "part by part." All we need is to sequentially store the finished part back to the secondary memory.
- We can conduct the following procedure:
  - Load $Q, K,$ and V to main memory.

Situation When Storing $T \times d$ Matrices Are Possible III

- For $i = 1, \ldots, T$, calculate

$$\mathbf{Q}_{i,:}\mathbf{K}^\top \in R^{1\times T}, \tag{10}$$

$$\text{SoftMax}(\mathbf{Q}_{i,:}\mathbf{K}^\top) \in R^{1\times T}, \tag{11}$$

$$\text{SoftMax}(\mathbf{Q}_{i,:}\mathbf{K}^\top)\mathbf{V} \in R^{1\times d} \tag{12}$$

and store the ith row of the output matrix to the secondary memory.

Situation When Storing $T \times d$ Matrices Are Possible IV

- By this way, the number of memory accesses is reduced to

$$O(Td).$$

because we never load/store any $T \times T$ matrices.

Situation When Storing $T \times d$ Matrices Are Not Possible I

- Unfortunately, our assumption of that $T \times d$ matrices can be stored in main memory is often untrue.
- We discuss the situation of

$$d \leq M \leq Td,$$

  where M is the size of the main memory.
- Then in (10)-(12), we must load the whole K and V once even for calculating just one output row.

Situation When Storing $T \times d$ Matrices Are Not Possible II

- A possible strategy is to calculate $|I|$ rows together:

$$\text{SoftMax}(\mathbf{Q}_{I,:}\mathbf{K}^\top)\mathbf{V}, \tag{13}$$

  where I is the block of rows that we intend to calculate.

- In calculating (13), we must access

  $|I|$ rows of Q, and the whole K and V (by row blocks).

Situation When Storing $T \times d$ Matrices Are Not Possible III

- We also need to store the intermediate block of

$$\mathbf{Q}_{I,:}\mathbf{K}^\top, \tag{14}$$

which requires

$$|I| \times T$$

space.

Situation When Storing $T \times d$ Matrices Are Not Possible IV

- The largest possible $|I|$ is

$$\frac{M}{T},$$

  where M is the size of the main memory.
- Thus the total number of memory accesses is

$$O(\frac{T}{M/T}) \times O(Td) = O(\frac{T^3 d}{M}). \quad (15)$$

Situation When Storing $T \times d$ Matrices Are Not Possible V

- In the above discussion, we see that the main bottleneck is to store the intermediate matrix in (14).
- Because the number of rows in (14) is a large number T, $|I|$ must be small. Thus we get a large first term in (15).

FlashAttention I

- To reduce the number of memory accesses, let us see if we may avoid storing the intermediate matrix in (14).
- Assume that we split Q to the following row-block form:

$$\begin{bmatrix} Q_{I_1,:} \\ \vdots \\ Q_{I_{Tr},:} \end{bmatrix}$$

with

$$|I_1| = \cdots = |I_{T_r}|.$$

FlashAttention II

- For K, V, we respectively split them to

$$\begin{bmatrix} K_{J_1,:} \\ \vdots \\ K_{J_{T_c},:} \end{bmatrix}, \begin{bmatrix} V_{J_1,:} \\ \vdots \\ V_{J_{T_c},:} \end{bmatrix}.$$

- We explain later why the split of Q should be different from that of K, V.
- In our discussion, we let I, J be any one of $|I_1|, \ldots, |I_{T_r}|$ and $|J_1|, \ldots, |J_{T_c}|$, respectively.

FlashAttention III

- We have

$$\text{SoftMax}(\mathbf{Q}_{I,:}\begin{bmatrix}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{K}_{J_{T_c},:}^\top\end{bmatrix})\begin{bmatrix}\mathbf{V}_{J_1,:}\\ \vdots \\ \mathbf{V}_{J_{T_c},:}\end{bmatrix}$$

$$=\text{SoftMax}(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_{T_c},:}^\top\end{bmatrix})\begin{bmatrix}\mathbf{V}_{J_1,:}\\ \vdots \\ \mathbf{V}_{J_{T_c},:}\end{bmatrix}$$

- If there is no SoftMax, we can see the result is

$$(\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top)\mathbf{V}_{J_1,:} + \cdots + (\mathbf{Q}_{I,:}\mathbf{K}_{J_{T_c},:}^\top)\mathbf{V}_{J_{T_c},:}.$$

FlashAttention IV

- We have

$$(\mathbf{Q}_{I,:}\mathbf{K}^\top_{J_1,:})\mathbf{V}_{J_1,:} \in |I| \times d,$$
$$\vdots$$
$$(\mathbf{Q}_{I,:}\mathbf{K}^\top_{J_{T_c},:})\mathbf{V}_{J_{T_c},:} \in |I| \times d.$$

- If we sequentially generate each term, there is no need to store the intermediate sub-matrix in (14).
- In this situation, the algorithm can be as follows.
- For $i = 1, \ldots, T_r$

FlashAttention V

- For $j = 1, \ldots, T_c$

$$O_{I_i,:} = O_{I_i,:} + (\mathbf{Q}_{I_i,:}\mathbf{K}_{J_j,:}^\top)\mathbf{V}_{J_j,:}. \tag{16}$$

  - Save $O_{I_i,:}$ to secondary memory.

- At any time point we must store the following four blocks in main memory
  1. $Q_{I,:} \in R^{|I| \times d}$
  2. $O_{I,:} \in R^{|I| \times d}$
  3. $K_{J,:} \in R^{|J| \times d}$ or $V_{J,:} \in R^{|J| \times d}$

     Note that we only need space for storing one of them. It can be overwritten once used.

FlashAttention VI

4. $\mathbf{Q}_{I,:}\mathbf{K}_{J,:}^\top \in R^{|I|\times|J|}$

- We assume that (16) does not require extra space to store the intermediate result $(\mathbf{Q}_{I_i,:}\mathbf{K}_{J_j,:}^\top)\mathbf{V}_{J_j,:}$. This is possible as any results can be immediately used to update O.
- Therefore, we select $|I|$ and $|J|$ to satisfy

$$|I|d < \frac{M}{4}, \quad |J|d < \frac{M}{4}, \quad |I||J| < \frac{M}{4}. \tag{17}$$

- We also need that

$$|I| \text{ as large as possible,}$$

FlashAttention VII

so K, V are less frequently loaded. The reasons is that for any I, we must access the whole K, V

- If we choose

$$|I| = \left\lfloor \frac{M}{4d} \right\rfloor \text{ and } |J| = \min(\left\lfloor \frac{M}{4d} \right\rfloor, d)$$

then (17) holds.

- In the end, the number of memory accesses is

$$O(\frac{T}{M/d}) \times O(Td) = O(\frac{T^2 d^2}{M}). \tag{18}$$

FlashAttention VIII

- This value is smaller than the one we had earlier in (15).
- Unfortunately, we need the whole intermediate matrix in (14) because the SoftMax function involves all elements in each row.
- A crucial observation is that we hope to have

FlashAttention IX

$$\mathsf{SoftMax}(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_j,:}^\top\end{bmatrix})\begin{bmatrix}\mathbf{V}_{J_1,:}\\ \vdots \\ \mathbf{V}_{J_j,:}\end{bmatrix} =$$

$$\Delta_{j-1} \odot (\mathsf{SoftMax}(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_{j-1},:}^\top\end{bmatrix})\begin{bmatrix}\mathbf{V}_{J_1,:}\\ \vdots \\ \mathbf{V}_{J_{j-1},:}\end{bmatrix})$$

$$+ \Delta_j \odot \left(\mathsf{SoftMax}(\mathbf{Q}_{I,:}\mathbf{K}_{J_j,:}^\top)\mathbf{V}_{J_j,:}\right), \tag{19}$$

FlashAttention X

where $\odot$ is the component-wise product and $\Delta_{j-1}, \Delta_j \in R^{|I|}$. If Δ_{j-1}, Δ_j are easily available, then we can do an operation similar to (16).

- We discuss why Δ_j can be easily calculated. While Δ_j is a vector, we give an illustration to get one of its components.
- We have $\forall t \in J_1 \cup \cdots \cup J_{j-1}$,

$$\frac{\exp(z_t)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp(z_s)} = \left(\frac{\sum_{s \in J_1 \cup \cdots \cup J_{j-1}} \exp(z_s)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp(z_s)} \right) \frac{\exp(z_t)}{\sum_{s \in J_1 \cup \cdots \cup J_{j-1}} \exp(z_s)},$$

FlashAttention XI

and $\forall t \in J_j$,

$$= \underbrace{\left(\frac{\sum_{s \in J_j} \exp(z_s)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp(z_s)} \right)}_{\Delta_j} \frac{\exp(z_t)}{\sum_{s \in J_j} \exp(z_s)}.$$

FlashAttention XII

- Clearly, all we need is to maintain

$$\sum_{s \in J_1 \cup \cdots \cup J_j} \exp(z_s). \tag{20}$$

- When handling j, we get

$$\sum_{s \in J_j} \exp(z_s),$$

FlashAttention XIII

so we can update (20) by

$$\sum_{s \in J_1 \cup \cdots \cup J_j} \exp(z_s)$$
$$= \sum_{s \in J_1 \cup \cdots \cup J_{j-1}} \exp(z_s) + \sum_{s \in J_j} \exp(z_s).$$

- This $O(|I|)$ cost for storing (20) in main memory is affordable.

FlashAttention XIV

- To have an algorithm, let's define

$$\bar{\Delta}_{\text{old}} = \exp(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_{j-1},:}^\top\end{bmatrix})\text{'s row sum}$$

$$\bar{\Delta} = \exp(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_j,:}^\top\end{bmatrix})\text{'s row sum}$$

$$\hat{\Delta} = \exp(\mathbf{Q}_{I,:}\mathbf{K}_{J_j,:}^\top)\text{'s row sum}$$

FlashAttention XV

- Then (19) becomes

$$\frac{\bar{\Delta}_{\text{old}}}{\bar{\Delta}} \odot (\text{SoftMax}(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_{j-1},:}^\top\end{bmatrix}) \begin{bmatrix}\mathbf{V}_{J_1,:} \\ \vdots \\ \mathbf{V}_{J_{j-1},:}\end{bmatrix})$$

$$+ \frac{\hat{\Delta}}{\bar{\Delta}} \odot \left(\text{SoftMax}(\mathbf{Q}_{I,:}\mathbf{K}_{J_j,:}^\top)\mathbf{V}_{J_j,:}\right),$$

$$= \frac{\bar{\Delta}_{\text{old}}}{\bar{\Delta}} \odot (\text{SoftMax}(\begin{bmatrix}\mathbf{Q}_{I,:}\mathbf{K}_{J_1,:}^\top & \cdots & \mathbf{Q}_{I,:}\mathbf{K}_{J_{j-1},:}^\top\end{bmatrix}) \begin{bmatrix}\mathbf{V}_{J_1,:} \\ \vdots \\ \mathbf{V}_{J_{j-1},:}\end{bmatrix})$$

$$+ \frac{1}{\bar{\Delta}} \odot \left(\exp(\mathbf{Q}_{I,:}\mathbf{K}_{J_j,:}^\top)\mathbf{V}_{J_j,:}\right).$$

FlashAttention XVI

- Finally, we have the following algorithm.

FlashAttention XVII

- For $i = 1, \ldots, T_r$
  - Load $\mathbf{Q}_{I_i,:}$
  - For $j = 1, \ldots, T_c$

$$\bar{\Delta}_{\text{old}} = \bar{\Delta}$$

$$\text{Load } \mathbf{K}_{J_j,:} \text{ and } P = \exp(\mathbf{Q}_{I_i,:}\mathbf{K}_{J_j,:}^\top) \quad (21)$$

$$\bar{\Delta} \leftarrow \bar{\Delta} + P\text{'s row sum}$$

$$\text{Load } \mathbf{V}_{J_j,:} \text{ to overwrite } \mathbf{K}_{J_j,:}$$

$$O_{I_i,:} = \frac{\bar{\Delta}_{\text{old}}}{\bar{\Delta}} \odot O_{I_i,:} + \frac{1}{\bar{\Delta}} \odot (P\mathbf{V}_{J_j,:}) \quad (22)$$

  - Save $O_{I_i,:}$ to secondary memory.

FlashAttention XVIII

- In (22), the fraction means component-wise division.
- For (21), we can calculate $\mathbf{Q}_{I_i,:}\mathbf{K}^\top_{J_j,:}$ first and calculate the exp value of each component. Thus, we only need space for one matrix.
- The number of memory accesses is the same as the algorithm analyzed in (16) and (18),

$$O(\frac{T^2d^2}{M}).$$

FlashAttention and FlashAttention-2 I

- FlashAttention was first introduced in Dao et al. (2022).
- Later Dao (2024) proposed an improved version FlashAttention-2.
- Interestingly, what we described earlier is FlashAttention-2.
- In the original FlashAttention, the algorithm is as follows

FlashAttention and FlashAttention-2 II

- For $j = 1, \ldots, T_c$
  - Load $\mathbf{V}_{J_j,:}$ and $\mathbf{K}_{J_j,:}$
  - For $i = 1, \ldots, T_r$

    Load $\bar{\Delta}_{I_i}, \mathbf{Q}_{I_i,:}$ and $P = \exp(\mathbf{Q}_{I_i,:}\mathbf{K}_{J_j,:}^\top)$

    $\bar{\Delta}_{\text{old}} = \bar{\Delta}_{I_i}$

    $\bar{\Delta}_{I_i} \leftarrow \bar{\Delta}_{I_i} + P$'s row sum

    Load $\mathbf{O}_{I_i,:}$ to overwrite $\mathbf{Q}_{I_i,:}$

    $$\mathbf{O}_{I_i,:} = \frac{\bar{\Delta}_{\text{old}}}{\bar{\Delta}} \odot \mathbf{O}_{I_i,:} + \frac{1}{\bar{\Delta}} \odot (P\mathbf{V}_{J_j,:})$$

    Save $\mathbf{O}_{I_i,:}, \bar{\Delta}_{I_i}$ to secondary memory

FlashAttention and FlashAttention-2 III

- A comparison with FlashAttention-2 shows that here i and j are swapped.
- Like FlashAttention-2, FlashAttention also maintain four matrices in the main memory
  1. $K_{J,:} \in R^{|J| \times d}$
  2. $V_{J,:} \in R^{|J| \times d}$
  3. $Q_{I,:} \in R^{|I| \times d}$ or $O_{I,:} \in R^{|I| \times d}$
     Note that we only need space for storing one of them. It can be overwritten once used.
  4. $\mathbf{Q}_{I,:}\mathbf{K}_{J,:}^\top \in R^{|I| \times |J|}$
- In the inner loop, we now need to

FlashAttention and FlashAttention-2 IV

  - load two matrices, and
  - save one matrix and one vector,

  from/to secondary memory, respectively.
- However, the earlier version (i.e., FlashAttention-2) needs to load only two matrices.
- The main issue is that FlashAttention sequentially uses

$$\mathbf{O}_{I_i,:}, \forall i$$

  in the inner loop, but we cannot afford to store them. Thus, we must save $\mathbf{O}_{I_i,:}$ back to secondary memory.

FlashAttention and FlashAttention-2 V

- Another issue is we must ensure that the save operation of $\mathbf{O}_{I_i,:}$ is finished before the load operation in iteration $i+1$. The reason is to free space for loading $\mathbf{Q}_{I_{i+1},:}$ or P.
- The above discussion explains why FlashAttention-2 is more memory efficient than FlashAttention.

Practical Implementation I

- The algorithm described above appear sufficient to perform the complete attention computation.
- Nevertheless, in practical implementations, the SoftMax function is computed in a modified form to ensure numerical stability.
- Specifically, instead of directly computing

$$\frac{\exp\left(z_t\right)}{\sum_{s=1}^{T}\exp\left(z_s\right)},$$

Practical Implementation II

the modified form below is commonly used,

$$\frac{\exp\left(z_t - m(z)\right)}{\sum_{s=1}^{T} \exp\left(z_s - m(z)\right)},$$

where

$$m(z) = \max_{1 \le s \le T} z_s.$$

Practical Implementation III

- Then, $\forall t \in J_1 \cup \cdots \cup J_{j-1}$,

$$\frac{\exp\left(z_t\right)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp\left(z_s\right)} = \frac{\exp\left(z_t - m(z)\right)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp\left(z_s - m(z)\right)}$$

and $\forall t \in J_j$,

$$\frac{\exp\left(z_t\right)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp\left(z_s\right)} = \frac{\exp\left(z_t - m(z)\right)}{\sum_{s \in J_1 \cup \cdots \cup J_j} \exp\left(z_s - m(z)\right)}$$

Practical Implementation IV

- Since $m(z)$ requires all elements in each row, it introduces a new challenge for our block-wise algorithm.
- Fortunately, a crucial observation is that $m(z)$ is decomposable by block, as the maximum operation is associative.
- Specifically,

$$\begin{aligned} m(z) &= \max_{1 \le s \le T} \; z_s, \\ &= \max_{s \in J_1 \cup \cdots \cup J_{Tc}} \; z_s, \\ &= \max \left\{ \max_{s \in J_1} \; z_s, \; \cdots, \; \max_{s \in J_{Tc}} \; z_s \right\}. \end{aligned}$$

Practical Implementation V

- Therefore, we have $\forall t \in J_1 \cup \cdots \cup J_{j-1}$,

$$
\begin{aligned}
&\exp\left(z_t - \max_{s \in J_1 \cup \cdots \cup J_j} z_s\right) \\
=&\exp\left(z_t - \max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s\right) \\
\times&\exp\left(\max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s - \max_{s \in J_1 \cup \cdots \cup J_j} z_s\right) \\
=&\exp\left(z_t - \max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s\right) \\
\times&\exp\left(\max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s - \max\left\{\max_{s \in J_j} z_s, \max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s\right\}\right)
\end{aligned}
$$

Practical Implementation VI

and $\forall t \in J_j$,

$$\exp\left(z_t - \max_{s \in J_1 \cup \cdots \cup J_j} z_s\right)$$

$$= \exp\left(z_t - \max_{s \in J_j} z_s\right) \times \exp\left(\max_{s \in J_j} z_s - \max_{s \in J_1 \cup \cdots \cup J_j} z_s\right)$$

$$= \exp\left(z_t - \max_{s \in J_j} z_s\right) \times \exp\left(\max_{s \in J_j} z_s - \max\left\{\max_{s \in J_j} z_s, \max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s\right\}\right)$$

Practical Implementation VII

- Like the sum of exponents in (20), all we need is to maintain

$$\max_{s \in J_1 \cup \cdots \cup J_j} z_s. \tag{23}$$

- When handling j, we get

$$\max_{s \in J_j} z_s,$$

Practical Implementation VIII

so we can update (23) by

$$\max_{s \in J_1 \cup \cdots \cup J_j} z_s$$
$$= \max\{\max_{s \in J_j} z_s, \max_{s \in J_1 \cup \cdots \cup J_{j-1}} z_s\}.$$

- Like (20), the cost for storing (23) is also $O(|I|)$ and affordable.
- By integrating the maintenance of (23) into our original algorithm, we can obtain a new algorithm that ensures numerical stability in practice.

References I

T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *Proceedings of The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.